Gaussian Process Forecasting

Kernels, posterior prediction, marginal likelihood and the `nabia` implementation

nabia documentation

October 2026

Abstract

This note documents `GaussianProcess` and the kernels in `Kernels.h`. We define Gaussian process regression on the time index and prove that the `nabia` kernels are valid covariance functions. The proofs cover the squared exponential kernel via its power series, the periodic kernel via a warping of time onto a circle, and sums and products via the Schur product theorem. We derive the gradient of every kernel with respect to its log-hyperparameters and the posterior predictive distribution. We also derive the log marginal likelihood and its gradient, using Jacobi's formula for the log-determinant. Further results: how normalizing the data changes the likelihood, the long-horizon behavior of forecasts (mean reversion for decaying kernels, exact periodicity for the periodic kernel), and the Cholesky-based algorithms with their costs. The last section maps everything to the code.

Contents

1 Model **1**

2 Kernels **2**
2.1 Definitions and log-parameter gradients 2
2.2 Validity 2

3 Posterior prediction **3**
3.1 Long-horizon behavior 3

4 Marginal likelihood **4**
4.1 Normalization 4

5 Computation **5**

6 Hyperparameter optimization **5**

7 Implementation **6**
7.1 Interface 6
7.2 Algorithm 6
7.3 Limitations 7

1 Model

Definition 1.1. A *Gaussian process* $f \sim \mathcal{GP}(0, k)$ on $\mathbb{R}$ is a random function such that, for any finite set of times $t_1, \dots, t_m$, the vector $(f(t_1), \dots, f(t_m))$ is Gaussian with mean zero and covariance $K_{ij} = k(t_i, t_j)$. A function k can serve as a covariance function iff it is symmetric and *positive semi-definite*: every such Gram matrix K is positive semi-definite.

`nabia` places the observations of a series at $t = 0, 1, \ldots, T-1$ and models

$$y_t = m + s\,(f(t) + \varepsilon_t), \qquad f \sim \mathcal{GP}(0, k), \qquad \varepsilon_t \overset{\text{iid}}{\sim} \mathcal{N}(0, \sigma_n^2), \tag{1}$$

where m is the sample mean and s the sample standard deviation (or $s = 1$ without normalization, Section 4.1). We work with the standardized series $z_t = (y_t - m)/s$, so that $z = f + \varepsilon$ with $z \sim \mathcal{N}(0, K)$, $K = K_f + \sigma_n^2 I$, and $(K_f)_{ij} = k(t_i, t_j)$.

The kernel encodes what the series looks like. Its variance sets the amplitude, its length scale sets how fast the function can change, and a period sets which patterns repeat. The data then choose these hyperparameters by maximizing the marginal likelihood (Section 4).

2 Kernels

All `nabia` kernels are stationary: they depend on $\tau = t - t'$ only. Their hyperparameters $\theta_j > 0$ are exposed as $\log \theta_j$ (see Section 6), and `gradient()` returns $\partial k / \partial \log \theta_j = \theta_j \, \partial k / \partial \theta_j$.

2.1 Definitions and log-parameter gradients

Squared exponential (`SquaredExponentialKernel`, hyperparameters σ^2, ℓ):

$$k(\tau) = \sigma^2 \exp\Big(-\frac{\tau^2}{2\ell^2}\Big), \qquad \frac{\partial k}{\partial \log \sigma^2} = k, \qquad \frac{\partial k}{\partial \log \ell} = k\,\frac{\tau^2}{\ell^2}.$$

The second derivative follows from $\partial(\tau^2/2\ell^2)/\partial\ell = -\tau^2/\ell^3$, multiplied by ℓ. Sample paths are infinitely differentiable.

Matérn 5/2 (`Matern52Kernel`, σ^2, ℓ): with $s = \sqrt{5}|\tau|/\ell$,

$$k = \sigma^2\Big(1 + s + \frac{s^2}{3}\Big)e^{-s}, \qquad \frac{\partial k}{\partial \log \sigma^2} = k, \qquad \frac{\partial k}{\partial \log \ell} = \sigma^2\,\frac{s^2(1+s)}{3}\,e^{-s}.$$

Derivation. $\frac{dk}{ds} = \sigma^2 e^{-s}\big[(1 + \frac{2s}{3}) - (1 + s + \frac{s^2}{3})\big] = -\sigma^2 e^{-s}\,\frac{s(1+s)}{3}$, and $\frac{ds}{d\log\ell} = -s$; multiply. Sample paths are twice differentiable. This kernel is usually a more realistic model than the squared exponential for economic and financial series.

Periodic (`PeriodicKernel`, σ^2, ℓ, p): with $u = \pi\tau/p$,

$$k = \sigma^2 \exp\Big(-\frac{2\sin^2 u}{\ell^2}\Big), \quad \frac{\partial k}{\partial \log \sigma^2} = k, \quad \frac{\partial k}{\partial \log \ell} = k\,\frac{4\sin^2 u}{\ell^2}, \quad \frac{\partial k}{\partial \log p} = k\,\frac{2u \sin 2u}{\ell^2}.$$

Derivation of the last one. $\frac{du}{d\log p} = -u$ and $\frac{d\sin^2 u}{du} = \sin 2u$, so $\frac{d\sin^2 u}{d\log p} = -u\sin 2u$. Then multiply by $-2k/\ell^2$.

Sum and product (`SumKernel`, `ProductKernel`): for $k = k_1 + k_2$ the gradient is the concatenation $(\nabla k_1, \nabla k_2)$. For $k = k_1 k_2$ the product rule gives $(k_2 \nabla k_1,\ k_1 \nabla k_2)$. Hyperparameters are ordered as those of k_1 followed by those of k_2, recursively.

2.2 Validity

Theorem 2.1 (Schur product theorem)**.** *If $A, B \in \mathbb{R}^{m\times m}$ are positive semi-definite, so is their elementwise product $A \circ B$.*

Proof. Write $B = \sum_k \lambda_k u_k u_k^\top$ with $\lambda_k \geq 0$. Then $(A \circ B)_{ij} = \sum_k \lambda_k (u_k)_i A_{ij} (u_k)_j$, so for any x, $x^\top (A \circ B) x = \sum_k \lambda_k \,(x \circ u_k)^\top A\,(x \circ u_k) \geq 0$. □

Corollary 2.2. *If k_1, k_2 are valid kernels, so are $k_1 + k_2$, $k_1 k_2$, $c\,k_1$ for $c > 0$, $k_1(\phi(t), \phi(t'))$ for any map ϕ, and pointwise limits of valid kernels.*

Proof. Gram matrices of $k_1 + k_2$ are sums of positive semi-definite matrices. Those of $k_1 k_2$ are Schur products (Theorem 2.1). A warped kernel's Gram matrix on $t_1, \ldots, t_m$ is k_1's Gram matrix on $\phi(t_1), \ldots, \phi(t_m)$. The positive semi-definite cone is closed, which covers limits. □

Proposition 2.3. *The squared exponential kernel is positive semi-definite, on $\mathbb{R}$ and on $\mathbb{R}^d$ with $\|t - t'\|$ in place of $|t - t'|$.*

Proof. On $\mathbb{R}$, write $e^{-(t-t')^2/2\ell^2} = g(t)\, g(t')\, e^{tt'/\ell^2}$ with $g(t) = e^{-t^2/2\ell^2}$. The power series $e^{tt'/\ell^2} = \sum_{n\geq 0}(tt')^n/(\ell^{2n} n!)$ is a limit of sums of the rank-one kernels $t^n t'^n$, which are positive semi-definite. Multiplying by $g(t)g(t')$ (a diagonal congruence DKD) preserves positive semi-definiteness, and so does scaling by $\sigma^2 > 0$. On $\mathbb{R}^d$, $e^{-\|t-t'\|^2/2\ell^2} = \prod_i e^{-(t_i - t'_i)^2/2\ell^2}$ is a product of valid kernels (Corollary 2.2). □

Proposition 2.4. *The periodic kernel is the squared exponential kernel on $\mathbb{R}^2$, with the same ℓ, applied to the warping $\phi(t) = (\cos(2\pi t/p), \sin(2\pi t/p))$. It is therefore positive semi-definite.*

Proof. $\|\phi(t) - \phi(t')\|^2 = 2 - 2\cos(2\pi(t-t')/p) = 4\sin^2(\pi(t-t')/p)$, so $\exp(-\|\phi(t) - \phi(t')\|^2/2\ell^2) = \exp(-2\sin^2(\pi\tau/p)/\ell^2)$. Conclude with Proposition 2.3 and Corollary 2.2. □

The Matérn 5/2 kernel is positive semi-definite by Bochner's theorem. Its spectral density is proportional to $(5/\ell^2 + \omega^2)^{-3}$, which is non-negative and integrable [1]. With $\sigma_n^2 > 0$, $K = K_f + \sigma_n^2 I$ is positive *definite* for every kernel above. Its Cholesky factor therefore exists in exact arithmetic. The floor `min_noise_variance` keeps it computable in floating point.

3 Posterior prediction

Theorem 3.1 (Predictive distribution)**.** *Let t_* be a vector of m new times, $K_* = k(t, t_*) \in \mathbb{R}^{T\times m}$ and $K_{**} = k(t_*, t_*)$. Under* (1)*, with $K = K_f + \sigma_n^2 I$,*

$$f(t_*) \mid z \sim \mathcal{N}(K_*^\top K^{-1} z,\ K_{**} - K_*^\top K^{-1} K_*), \qquad z_* \mid z \sim \mathcal{N}\big(K_*^\top K^{-1} z,\ K_{**} - K_*^\top K^{-1} K_* + \sigma_n^2 I\big).$$

Proof. $(z, f(t_*))$ is jointly Gaussian with zero mean, $\mathrm{Cov}(z) = K$, $\mathrm{Cov}(f(t_*)) = K_{**}$ and $\mathrm{Cov}(z, f(t_*)) = \mathrm{Cov}(f(t), f(t_*)) = K_*$, because the noise is independent of f. The first claim is the conditioning lemma (proved in the Kalman filter note, Lemma 2.1). A new observation $z_* = f(t_*) + \varepsilon_*$ adds independent noise, which adds $\sigma_n^2 I$ to the covariance. □

The predictive mean $K_*^\top \alpha$, with $\alpha = K^{-1} z$, is a weighted sum of kernel functions centred at the data. The variance is the prior variance minus what the data explain. `GaussianProcess::predict` returns, for each t_*, the marginal mean and variance of $y_* = m + s\, z_*$ (noise included), and `forecast(H)` calls it with $t_* = T, \ldots, T + H - 1$.

3.1 Long-horizon behavior

Proposition 3.2 (Mean reversion)**.** *If $k(\tau) \to 0$ as $|\tau| \to \infty$ (squared exponential, Matérn), then as $t_* \to \infty$ the predictive mean of y_* tends to m and its variance to $s^2(k(0) + \sigma_n^2)$.*

Proof. Each entry of K_* is $k(t_i - t_*) \to 0$, while K^{-1} and z are fixed. So the mean $K_*^\top K^{-1} z \to 0$ and the variance $k(0) - K_*^\top K^{-1} K_* + \sigma_n^2 \to k(0) + \sigma_n^2$. Transform back with $y_* = m + s z_*$. □

Proposition 3.3 (Periodic extrapolation)**.** *For the periodic kernel with period p, the predictive mean and variance at $t_* + p$ equal those at t_*.*

Proof. $k(t_i, t_* + p) = k(t_i, t_*)$ and $k(t_* + p, t_* + p) = k(t_*, t_*)$, since $\sin^2$ has period π. Hence K_* and K_{**} are unchanged, and so is everything in Theorem 3.1. □

So a decaying kernel is a local model. Its forecasts fade towards the mean on the time scale of the length scale, and its intervals widen to the unconditional spread. Long-range structure must be built into the kernel: a periodic component (or a sum with one) repeats seasonality indefinitely, and a product of periodic and decaying kernels lets the pattern drift.

4 Marginal likelihood

Integrating out f, $z \sim \mathcal{N}(0, K(\theta))$, so the log marginal likelihood is

$$\log p(z \mid \theta) = -\tfrac{1}{2}\, z^\top K^{-1} z \;-\; \tfrac{1}{2} \log\det K \;-\; \tfrac{T}{2} \log 2\pi. \tag{2}$$

The first term rewards fitting the data. The second penalizes complexity: a flexible kernel (short length scales, small noise) can explain many different data sets, so it spreads its probability mass thinly and pays through a larger $\log\det K$. The trade-off is automatic, an Occam's razor that needs no validation set.

Lemma 4.1. *For a smooth family of invertible symmetric matrices $K(\theta)$, with $K' = \partial K/\partial\theta$: (i) $\partial K^{-1}/\partial\theta = -K^{-1}K'K^{-1}$; (ii) $\partial \log\det K/\partial\theta = \operatorname{tr}(K^{-1}K')$ when $K \succ 0$.*

Proof. (i) Differentiate $KK^{-1} = I$: $K'K^{-1} + K\,\partial K^{-1} = 0$. (ii) $\det(K + \epsilon K') = \det K\, \det(I + \epsilon K^{-1}K')$, and $\det(I + \epsilon A) = \prod_i (1 + \epsilon\lambda_i(A)) = 1 + \epsilon \operatorname{tr} A + O(\epsilon^2)$. Hence $\log\det(K + \epsilon K') = \log\det K + \epsilon \operatorname{tr}(K^{-1}K') + O(\epsilon^2)$. □

Theorem 4.2 (Gradient of the log marginal likelihood)**.** *With $\alpha = K^{-1}z$,*

$$\frac{\partial \log p(z \mid \theta)}{\partial \theta_j} = \tfrac{1}{2}\, \alpha^\top \frac{\partial K}{\partial \theta_j} \alpha - \tfrac{1}{2} \operatorname{tr}\Big(K^{-1} \frac{\partial K}{\partial \theta_j}\Big) = \tfrac{1}{2} \operatorname{tr}\Big((\alpha\alpha^\top - K^{-1})\, \frac{\partial K}{\partial \theta_j}\Big).$$

Proof. Differentiate (2) with Lemma 4.1: $\partial(z^\top K^{-1} z) = -z^\top K^{-1}K'K^{-1}z = -\alpha^\top K'\alpha$ and $\partial \log\det K = \operatorname{tr}(K^{-1}K')$. Finally, $\alpha^\top K'\alpha = \operatorname{tr}(\alpha\alpha^\top K')$. □

For a kernel hyperparameter, $\partial K/\partial \log\theta_j$ is the matrix of `gradient()` entries. For the noise, $\partial K/\partial \log\sigma_n^2 = \sigma_n^2 I$, so $\partial \log p/\partial \log\sigma_n^2 = \tfrac{1}{2}\sigma_n^2 \operatorname{tr}(\alpha\alpha^\top - K^{-1})$.

4.1 Normalization

Proposition 4.3. *If $z = (y - m\mathbf{1})/s$ with fixed m, s, then $\log p(y) = \log p(z) - T \log s$.*

Proof. The map $y \mapsto z$ is affine with Jacobian s^{-T}, so $p_y(y) = p_z(z)\, s^{-T}$. □

`nabia` fits on z when `normalize` is true (the default), so that unit variances and noise levels around 10^{-2} are sensible starting points whatever the units of y. The constant $-T \log s$ does not depend on θ, so the optimum is unchanged. `log_marginal_likelihood()` reports $\log p(y)$ in the units of y, using Proposition 4.3. Fitted kernel variances and the noise variance are in normalized units: multiply by s^2 for the units of y. The mean m is the sample mean, plugged in rather than estimated jointly. This is standard and harmless when the series has no strong trend.

5 Computation

Cholesky factorization. Write $K = LL^\top$ with L lower triangular (cost $T^3/3$). Then:

- $\alpha = L^{-\top}(L^{-1}z)$, two triangular solves costing $O(T^2)$;
- $\log\det K = 2\sum_i \log L_{ii}$, since $\det K = \det(L)^2 = \prod_i L_{ii}^2$;
- for predictions, $v = L^{-1}K_*$ gives $K_*^\top K^{-1} K_* = v^\top v$, so the predictive variances are $\mathrm{diag}(K_{**}) - \sum_i v_i^2$. This costs $O(T^2)$ per new time and never forms K^{-1}.

Subtracting $v^\top v$ can produce tiny negative values in floating point; `predict` clamps the latent variance at 0 before adding the noise.

Gradient. Theorem 4.2 needs $W = \alpha\alpha^\top - K^{-1}$. `nabia` forms K^{-1} from the Cholesky factor ($O(T^3)$) and accumulates $-\frac{1}{2}\sum_{ij} W_{ij}\,\partial k(t_i,t_j)/\partial \log\theta$ for all hyperparameters in one pass over the T^2 pairs. One evaluation of the objective and its gradient therefore costs $O(T^3 + T^2 n_\theta)$.

Operation	Cost	Memory
Gram matrix	$O(T^2)$ kernel evaluations	$O(T^2)$
Cholesky, α, $\log\det$	$O(T^3)$	$O(T^2)$
Likelihood gradient	$O(T^3 + T^2 n_\theta)$	$O(T^2)$
Prediction at m times	$O(T^2 m)$	$O(Tm)$

In practice this means a few thousand observations at most. Beyond that, state-space representations of Matérn kernels (which reduce to Kalman filtering) or sparse approximations are needed.

6 Hyperparameter optimization

The hyperparameters are $\theta = (\log\theta_1, \ldots, \log\theta_{n_k}, \log\sigma_n^2)$, without the last entry when `optimize_noise` is false. `fit` minimizes $-\log p(z \mid \theta)$ with `LBFGS` (tolerance 10^{-5} on the gradient, at most `max_iter` iterations), starting from the kernel and noise given in the parameters.

- *Why log-parameters.* Positivity becomes unconstrained, and the objective is much better conditioned. Length scales and variances matter on a relative scale, so a unit step in $\log\ell$ means the same thing for $\ell = 0.1$ and $\ell = 100$.
- *Safeguards.* A trial with $|\theta_j| > 20$ (a factor $e^{20} \approx 5 \cdot 10^8$), or whose K fails the Cholesky factorization, returns $+\infty$. The L-BFGS line search rejects such steps and backtracks.
- *Noise floor.* The noise is clamped at `min_noise_variance`. In the clamped region the true gradient in $\log\sigma_n^2$ is zero, but the formula above is not. This only matters if the optimum lies at the floor, which signals noiseless data.
- *Local optima.* The marginal likelihood is not concave and can be multimodal: for example, a short-length-scale explanation that fits the noise against a long-length-scale one with more noise, or periods that are multiples of each other. Start from plausible values, such as a period near the known seasonality. The `nabia` tests start a periodic kernel at $p = 11.5$ and recover $p = 12.0$. Restarting from several initial points is a cheap safeguard.
- The parameter struct keeps the initial kernel. The fitted copy is available through `kernel()`, and `noise_variance()` returns the fitted noise.

7 Implementation

7.1 Interface

```
auto p = std::make_unique<GaussianProcessParams>();
p->kernel = std::make_unique<SumKernel>(
    std::make_unique<Matern52Kernel>(1.0, 20.0), // slow trend
    std::make_unique<PeriodicKernel>(0.5, 1.0, 12.0)); // seasonality
p->noise_variance = 1e-2; // initial value
GaussianProcess gp(std::move(p));
gp.fit(y); // equally spaced, finite values
Forecast fc = gp.forecast(24); // t = T .. T+23, noise included
Forecast at = gp.predict(times); // any times, e.g. between observations
double lml = gp.log_marginal_likelihood();
```

7.2 Algorithm

Algorithm 1 `GaussianProcess::fit` and `predict`

Require: $y_0, \dots, y_{T-1}$ finite, $T \geq 2$; kernel k; noise σ_n^2

$t \leftarrow (0, \dots, T-1)$; $m \leftarrow \bar{y}$; $s \leftarrow$ sample sd (or 1); $z \leftarrow (y - m)/s$

if `optimize_hyperparameters` **then**

  $\theta^\star \leftarrow$ L-BFGS minimizer of $-\log p(z \mid \theta)$, using Theorem 4.2 for the gradient

  set k and σ_n^2 from $\theta^\star$

end if

$L \leftarrow \mathrm{chol}(K_f + \sigma_n^2 I)$; $\alpha \leftarrow L^{-\top} L^{-1} z$ ▷ stored for prediction

predict(t_*):

$K_* \leftarrow k(t, t_*)$; $V \leftarrow L^{-1} K_*$

$\mu \leftarrow m + s\, K_*^\top \alpha$; $v \leftarrow s^2 \big(\max(\mathrm{diag}\, K_{**} - \mathrm{colsum}(V \circ V), 0) + \sigma_n^2 \big)$

Validation. The constructor requires a kernel and positive noise variances. `fit` requires at least two finite observations, so the GP does not accept missing values; use the Kalman filter for gappy series. Kernels reject non-positive hyperparameters. A final Cholesky failure, which only happens if the noise is too small for the kernel, throws `std::invalid_argument`.

Correspondence with the code.

Mathematics	`GaussianProcess` / kernels
$k(t, t')$, $\partial k / \partial \log \theta$	`IKernel::operator()`, `gradient()`
$\log \theta$	`log_hyperparameters()`, `set_log_hyperparameters()`
K_f	`IKernel::matrix(t, t)`
m, s	private `offset_`, `scale_`
L, α	private `chol_`, `alpha_`
$\log p(y)$	`log_marginal_likelihood()`
$m + sK_f\alpha$	`fitted_values()`

Tests. `tests/test_forecasters.cpp` checks:

- every kernel's gradient against central finite differences, plus symmetry;
- interpolation of a smooth function, with error below 10^{-3} between the data points;

- that hyperparameter fitting raises the log marginal likelihood by more than 10 nats and finds a plausible length scale;
- Proposition 3.3: a periodic kernel recovers period 12.0, extrapolates 24 steps to within 0.1, and the true values fall inside the 95% intervals;
- Proposition 3.2: at $t_* = 1000$ the forecast equals m and its variance equals $s^2(k(0) + \sigma_n^2)$, both to 10^{-12};
- rejection of invalid input.

7.3 Limitations

Cost is cubic in T. Inputs are equally spaced time indices, although `predict` accepts any times. The mean is constant, the noise Gaussian and homoscedastic, and the kernels stationary. There are no automatic restarts in the hyperparameter search.

References

[1] C. E. Rasmussen and C. K. I. Williams, *Gaussian Processes for Machine Learning*, MIT Press, 2006.

[2] S. Roberts, M. Osborne, M. Ebden, S. Reece, N. Gibson and S. Aigrain, Gaussian processes for time-series modelling, *Phil. Trans. R. Soc. A* 371 (2013) 20110550.

[3] D. J. C. MacKay, Introduction to Gaussian processes, in C. M. Bishop (ed.), *Neural Networks and Machine Learning*, Springer, 1998, 133–165.

[4] R. A. Horn and C. R. Johnson, *Matrix Analysis*, 2nd ed., Cambridge University Press, 2013.

[5] J. Hartikainen and S. Särkkä, Kalman filtering and smoothing solutions to temporal Gaussian process regression models, *IEEE MLSP* (2010) 379–384.